

Lec. 2

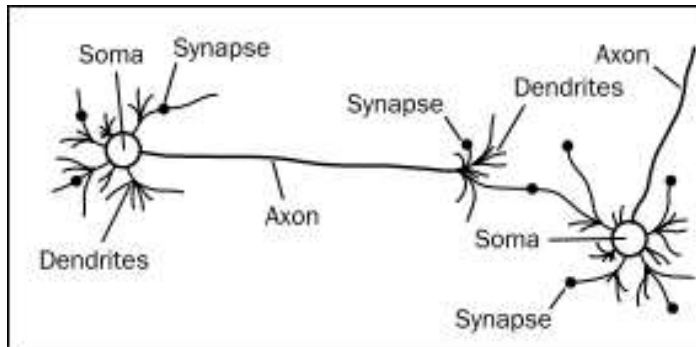


Table 6.1 Analogy between biological and artificial neural networks

Biological neural network	Artificial neural network
Soma	Neuron
Dendrite	Input
Axon	Output
Synapse	Weight

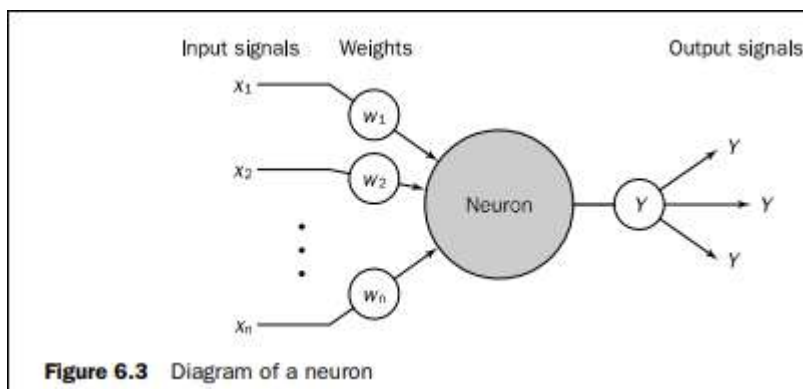


Figure 6.3 Diagram of a neuron

$$X = \sum_{i=1}^n x_i w_i$$

$$Y = \begin{cases} +1 & \text{if } X \geq \theta \\ -1 & \text{if } X < \theta \end{cases}$$

$$Y = \text{sign} \left[\sum_{i=1}^n x_i w_i - \theta \right]$$

McCulloch-Pitts Neuron Model

The first formal definition of a synthetic neuron model based on the highly simplified considerations of the biological model described in the preceding section was formulated by McCulloch and Pitts (1943). The McCulloch-Pitts model of the neuron is shown in Figure 2.3a. The inputs x_i , for $i = 1, 2, \dots, n$, are 0 or 1, depending on the absence or presence of the input impulse at instant k . The neuron's output signal is denoted as o . The firing rule for this model is defined as follows

$$o^{k+1} = \begin{cases} 1 & \text{if } \sum_{i=1}^n w_i x_i^k \geq T \\ 0 & \text{if } \sum_{i=1}^n w_i x_i^k < T \end{cases}$$

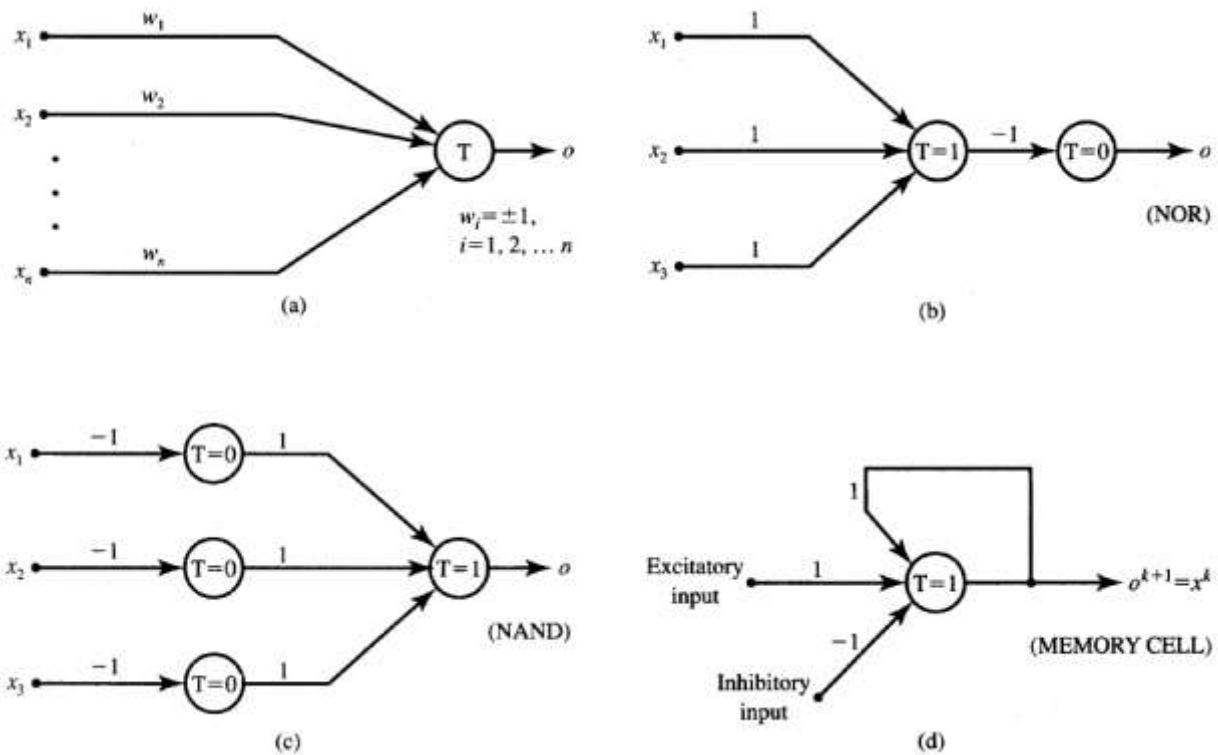


Figure 2.3 McCulloch-Pitts model neuron and elementary logic networks: (a) model diagram, (b) NOR gate, (c) NAND gate, and (d) memory cell.

Also, the steps in this method are very similar to how Neural Networks learn, which is as follows;

- Initialize weight values and bias
- Forward Propagate
- Check the error
- Backpropagate and Adjust weights and bias
- Repeat for all training examples

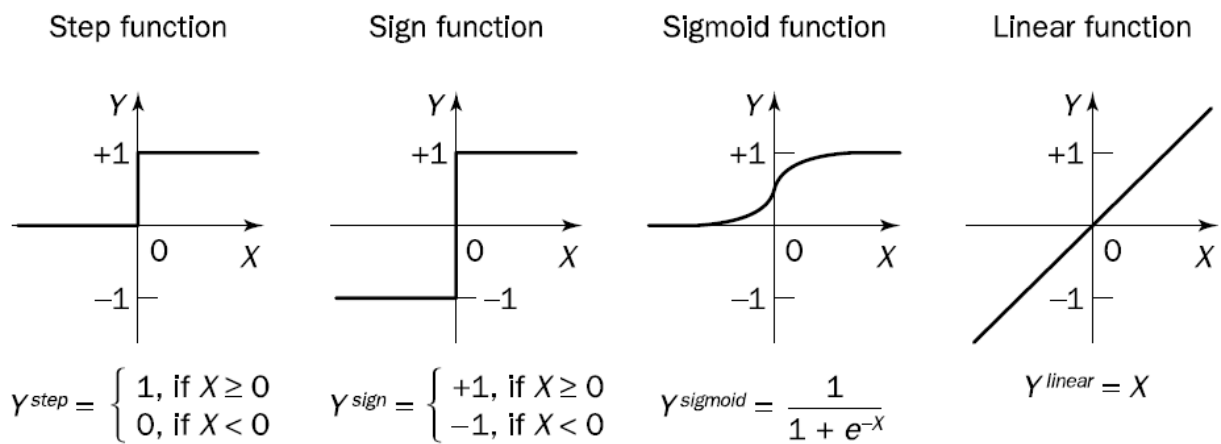


Figure 6.4 Activation functions of a neuron

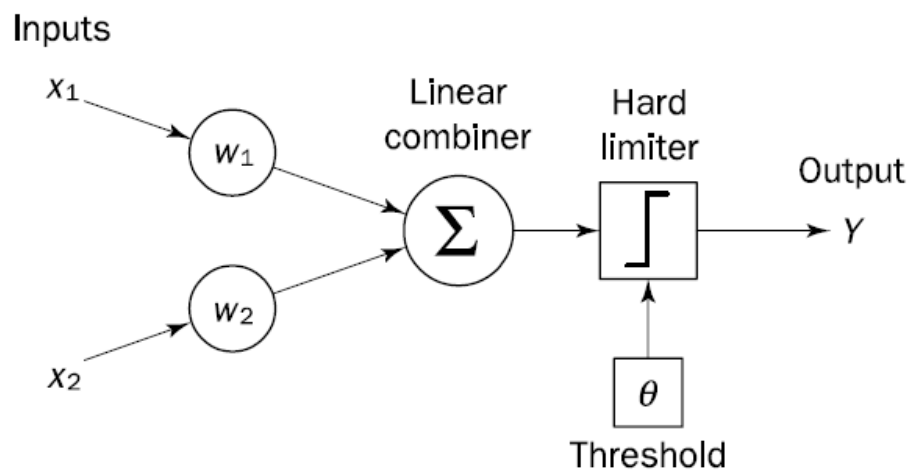
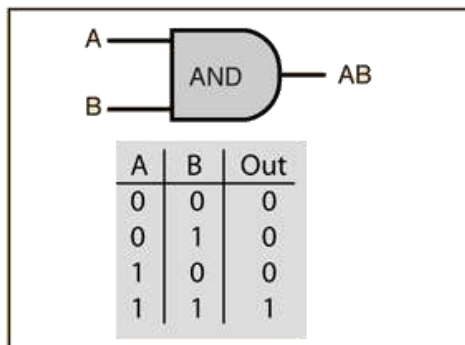


Figure 6.5 Single-layer two-input perceptron

Neural Representation of AND, OR, NOT, XOR and XNOR Logic Gates (Perceptron Algorithm)

AND Gate

From our knowledge of logic gates, we know that an AND logic table is given by the diagram below



The question is, what are the weights and bias for the AND perceptron?

First, we need to understand that the output of an AND gate is 1 only if both inputs (in this case, x_1 and x_2) are 1. So, following the steps listed above;

Row 1

- From $w_1 \cdot x_1 + w_2 \cdot x_2 + b$, initializing w_1 , w_2 , as 1 and b as -1 , we get;
 $x_1(1) + x_2(1) - 1$

- Passing the first row of the AND logic table ($x_1=0$, $x_2=0$), we get;
 $0 + 0 - 1 = -1$

- From the Perceptron rule, if $Wx + b \leq 0$, then $y' = 0$. Therefore, this row is correct, and no need for Backpropagation.

Row 2

- Passing ($x_1=0$ and $x_2=1$), we get;
 $0 + 1 - 1 = 0$

- From the Perceptron rule, if $Wx + b \leq 0$, then $y' = 0$. This row is correct, as the output is 0 for the AND gate.
- From the Perceptron rule, this works (for both row 1, row 2 and 3).

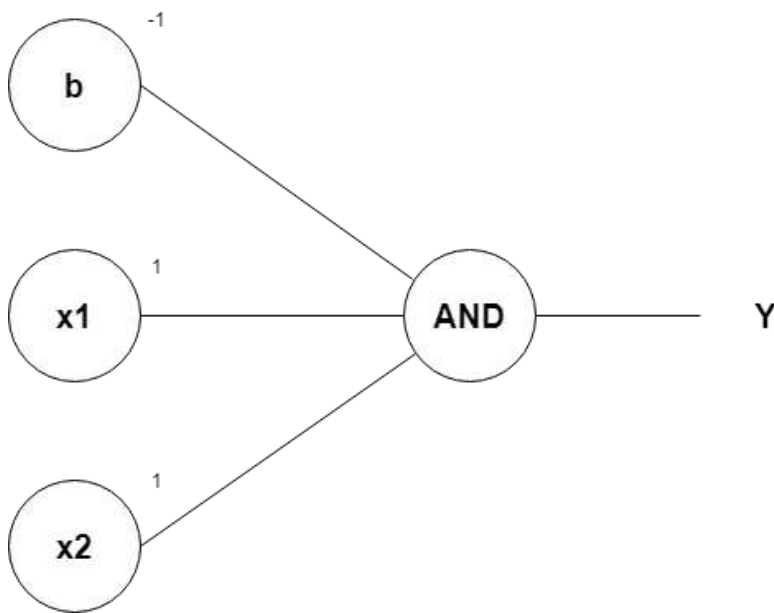
Row 4

- Passing ($x_1=1$ and $x_2=1$), we get;
 $1 + 1 - 1 = 1$

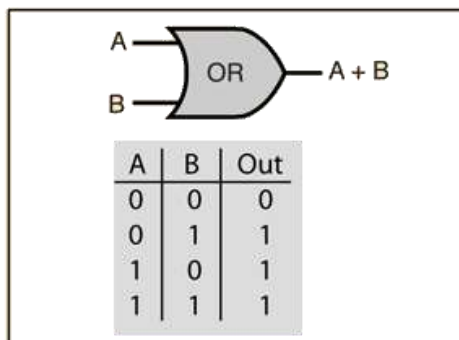
- Again, from the perceptron rule, this is still valid.

Therefore, we can conclude that the model to achieve an AND gate, using the Perceptron algorithm is;

$$x_1 + x_2 - 1$$



OR Gate



From the diagram, the OR gate is 0 only if both inputs are 0.

Row 1

- From $w_1x_1 + w_2x_2 + b$, initializing w_1, w_2 , as 1 and b as -1 , we get;

$$x_1(1) + x_2(1) - 1$$

- Passing the first row of the OR logic table ($x_1=0, x_2=0$), we get;

$$0 + 0 - 1 = -1$$

- From the Perceptron rule, if $Wx + b \leq 0$, then $y' = 0$. Therefore, this row is correct.

Row 2

- Passing ($x_1=0$ and $x_2=1$), we get;

$$0 + 1 - 1 = 0$$

- From the Perceptron rule, if $Wx + b \leq 0$, then $y' = 0$. Therefore, this row is incorrect.
- So we want values that will make inputs $x_1=0$ and $x_2=1$ give y' a value of 1. If we change w_2 to 2, we have;

$$0 + 2 - 1 = 1$$

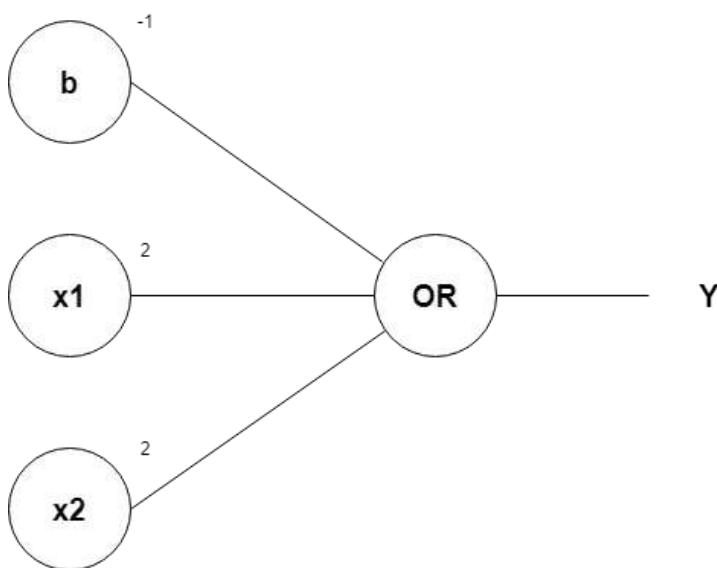
- From the Perceptron rule, this is correct for both the row 1 and 2.

Row 3

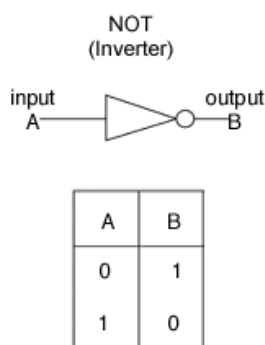
- Passing ($x_1=1$ and $x_2=0$), we get;
 $1+0-1 = 0$
- From the Perceptron rule, if $Wx+b \leq 0$, then $y^*=0$. Therefore, this row is incorrect.
- Since it is similar to that of row 2, we can just change w_1 to 2, we have;
 $2+0-1 = 1$
- From the Perceptron rule, this is correct for both the row 1, 2 and 3.

Row 4

- Passing ($x_1=1$ and $x_2=1$), we get;
 $2+2-1 = 3$
 - Again, from the perceptron rule, this is still valid. Quite Easy!
- Therefore, we can conclude that the model to achieve an OR gate, using the Perceptron algorithm is;
 $2x_1+2x_2-1$



NOT Gate



From the diagram, the output of a NOT gate is the inverse of a single input. So, following the steps listed above;

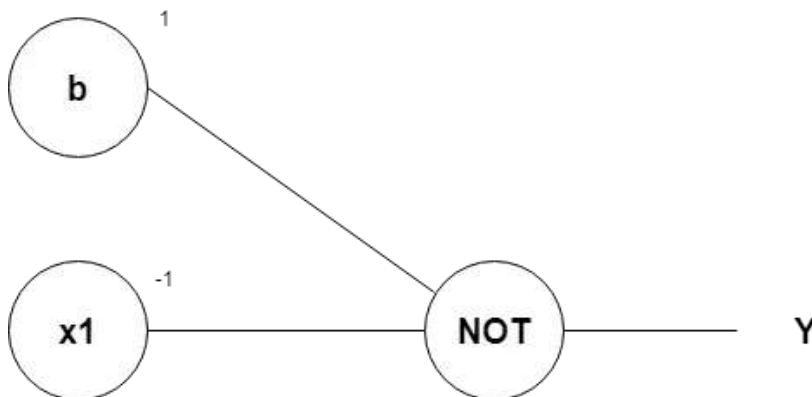
Row 1

- From w_1x_1+b , initializing w_1 as 1 (since single input), and b as -1 , we get;
 $x_1(1)-1$
- Passing the first row of the NOT logic table ($x_1=0$), we get;
 $0-1 = -1$
- From the Perceptron rule, if $Wx+b \leq 0$, then $y'=0$. This row is incorrect, as the output is 1 for the NOT gate.
- So we want values that will make input $x_1=0$ to give y' a value of 1. If we change b to 1, we have;
 $0+1 = 1$
- From the Perceptron rule, this works.

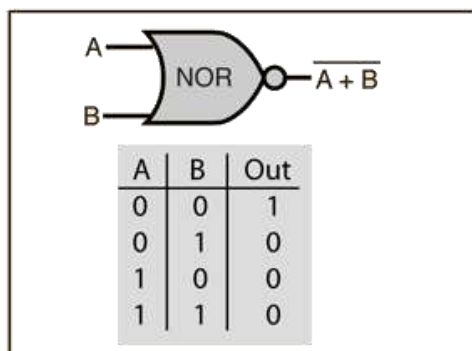
Row 2

- Passing ($x_1=1$), we get;
 $1+1 = 2$
- From the Perceptron rule, if $Wx+b > 0$, then $y'=1$. This row is so incorrect, as the output is 0 for the NOT gate.
- So we want values that will make input $x_1=1$ to give y' a value of 0. If we change w_1 to -1 , we have;
 $-1+1 = 0$
- From the Perceptron rule, if $Wx+b \leq 0$, then $y'=0$. Therefore, this works (for both row 1 and row 2).

Therefore, we can conclude that the model to achieve a NOT gate, using the Perceptron algorithm is;
 $-x_1+1$



NOR Gate



From the diagram, the NOR gate is 1 only if both inputs are 0.

Row 1

- From $w_1x_1 + w_2x_2 + b$, initializing w_1 and w_2 as 1, and b as -1, we get;

$$x_1(1) + x_2(1) - 1$$

- Passing the first row of the NOR logic table ($x_1=0, x_2=0$), we get;

$$0+0-1 = -1$$

- From the Perceptron rule, if $Wx+b \leq 0$, then $y'=0$. This row is incorrect, as the output is 1 for the NOR gate.
- So we want values that will make input $x_1=0$ and $x_2 = 0$ to give y' a value of 1. If we change b to 1, we have;

$$0+0+1 = 1$$

- From the Perceptron rule, this works.

Row 2

- Passing ($x_1=0, x_2=1$), we get;

$$0+1+1 = 2$$

- From the Perceptron rule, if $Wx+b > 0$, then $y'=1$. This row is incorrect, as the output is 0 for the NOR gate.
- So we want values that will make input $x_1=0$ and $x_2 = 1$ to give y' a value of 0. If we change w_2 to -1, we have;

$$0-1+1 = 0$$

- From the Perceptron rule, this is valid for both row 1 and row 2.

Row 3

- Passing ($x_1=1, x_2=0$), we get;

$$1+0+1 = 2$$

- From the Perceptron rule, if $Wx+b > 0$, then $y'=1$. This row is incorrect, as the output is 0 for the NOR gate.
- So we want values that will make input $x_1=0$ and $x_2 = 1$ to give y' a value of 0. If we change w_1 to -1, we have;

$$-1+0+1 = 0$$

- From the Perceptron rule, this is valid for both row 1, 2 and 3.

Row 4

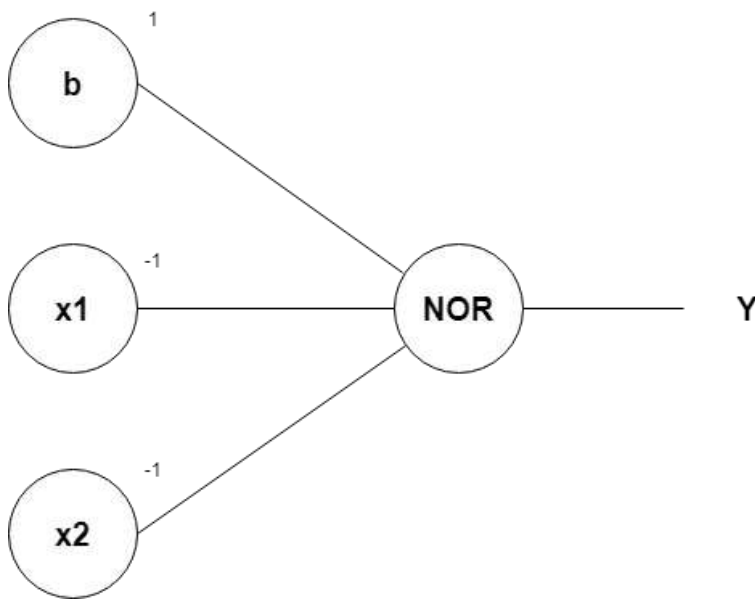
- Passing ($x_1=1, x_2=1$), we get;

$$-1-1+1 = -1$$

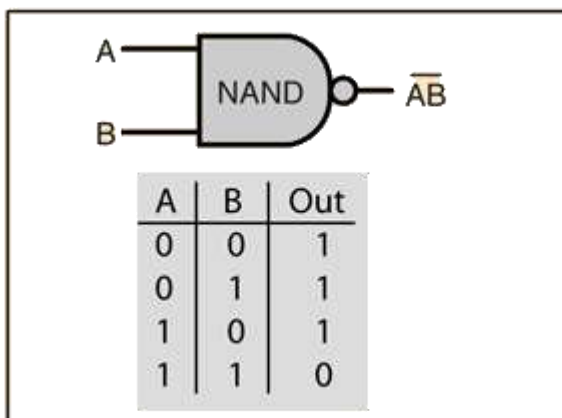
- From the Perceptron rule, this still works.

Therefore, we can conclude that the model to achieve a NOR gate, using the Perceptron algorithm is;

$$-x_1 - x_2 + 1$$



NAND Gate



From the diagram, the NAND gate is 0 only if both inputs are 1.

Row 1

- From $w_1x_1 + w_2x_2 + b$, initializing w_1 and w_2 as 1, and b as -1, we get;
 $x_1(1) + x_2(1) - 1$

- Passing the first row of the NAND logic table ($x_1=0, x_2=0$), we get;
 $0 + 0 - 1 = -1$

- From the Perceptron rule, if $Wx + b \leq 0$, then $y' = 0$. This row is incorrect, as the output is 1 for the NAND gate.
- So we want values that will make input $x_1=0$ and $x_2 = 0$ to give y' a value of 1. If we change b to 1, we have;

$$0 + 0 + 1 = 1$$

- From the Perceptron rule, this works.

Row 2

- Passing ($x_1=0, x_2=1$), we get;

$$0+1+1 = 2$$

- From the Perceptron rule, if $Wx+b > 0$, then $y'=1$. This row is also correct (for both row 2 and row 3).

Row 4

- Passing $(x1=1, x2=1)$, we get;

$$1+1+1 = 3$$

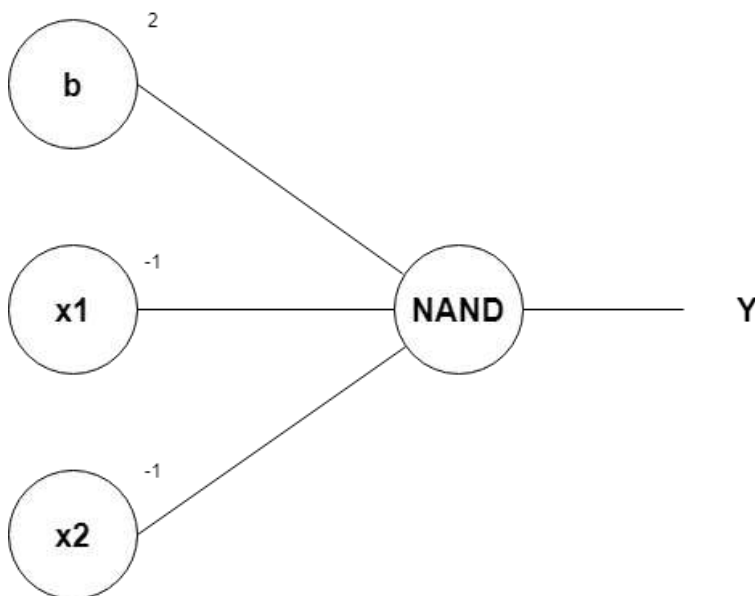
- This is not the expected output, as the output is 0 for a NAND combination of $x1=1$ and $x2=1$.
- Changing values of $w1$ and $w2$ to -1, and value of b to 2, we get;

$$-1-1+2 = 0$$

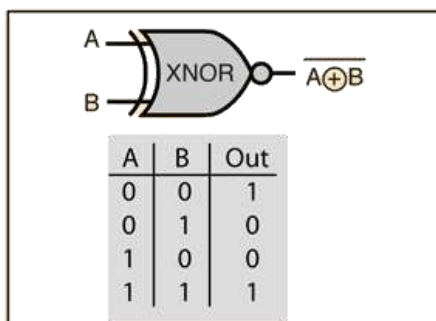
- It works for all rows.

Therefore, we can conclude that the model to achieve a NAND gate, using the Perceptron algorithm is;

$$-x1-x2+2$$



XNOR Gate



Now that we are done with the necessary basic logic gates, we can combine them to give an XNOR gate.

The boolean representation of an XNOR gate is;

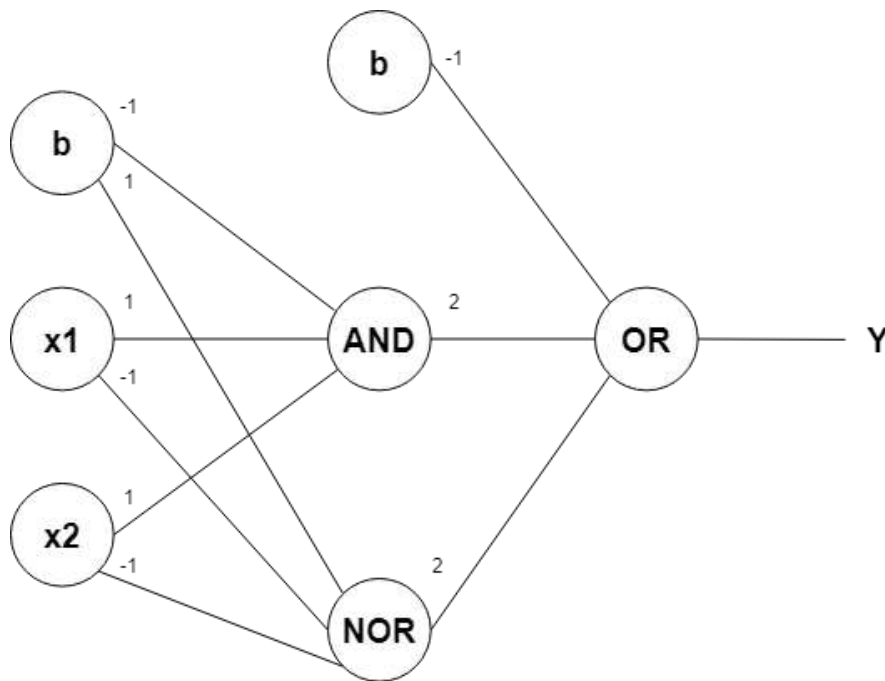
$$x_1x_2 + x_1x_2'$$

Where ' means inverse.

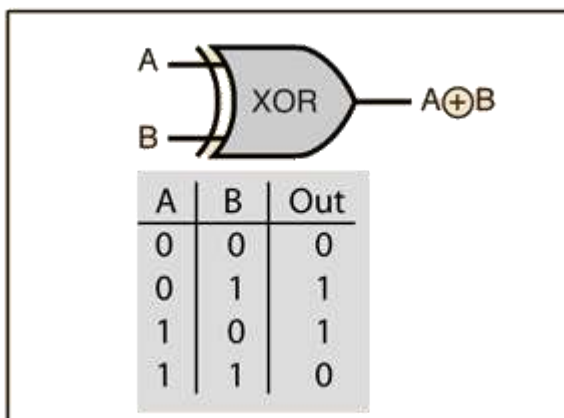
From the expression, we can say that the XNOR gate consists of an AND gate (x_1x_2), a NOR gate (x_1x_2'), and an OR gate.

This means we will have to combine 3 perceptrons:

- AND (x_1+x_2-1)
- NOR ($-x_1-x_2+1$)
- OR ($2x_1+2x_2-1$)



XOR Gate



The boolean representation of an XOR gate is;

$$x_1x_2 + x_1x_2$$

We first simplify the boolean expression

$$x_1x_2 + x_1x_2 + x_1x_1 + x_2x_2$$

$$x_1(x_1 + x_2) + x_2(x_1 + x_2)$$

$$(x_1 + x_2)(x_1 + x_2)$$

$$(x_1 + x_2)(x_1x_2)$$

From the simplified expression, we can say that the XOR gate consists of an OR gate ($x_1 + x_2$), a NAND gate ($-x_1-x_2+1$) and an AND gate ($x_1+x_2-1.5$).

This means we will have to combine 2 perceptrons:

- OR ($2x_1+2x_2-1$)
- NAND ($-x_1-x_2+2$)
- AND (x_1+x_2-1)

The boolean representation of an XOR gate is;

$$x_1x_2 + x_1x_2$$

We first simplify the boolean expression

$$x_1x_2 + x_1x_2 + x_1x_1 + x_2x_2$$

$$x_1(x_1 + x_2) + x_2(x_1 + x_2)$$

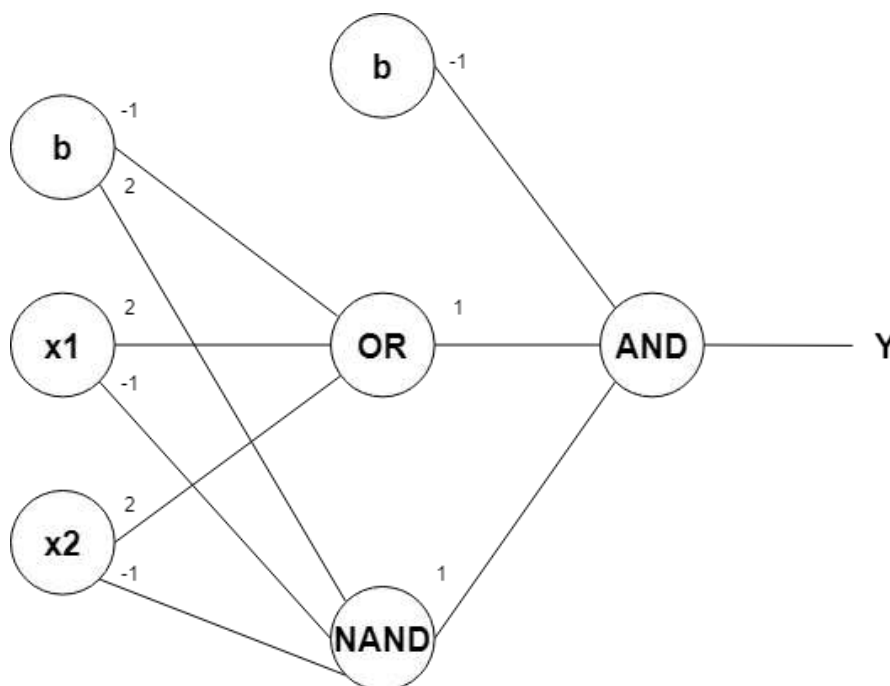
$$(x_1 + x_2)(x_1 + x_2)$$

$$(x_1 + x_2)(x_1x_2)$$

From the simplified expression, we can say that the XOR gate consists of an OR gate ($x_1 + x_2$), a NAND gate ($-x_1-x_2+1$) and an AND gate ($x_1+x_2-1.5$).

This means we will have to combine 2 perceptrons:

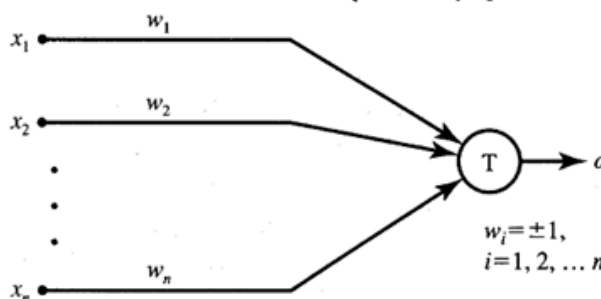
- OR ($2x_1+2x_2-1$)
- NAND ($-x_1-x_2+2$)
- AND (x_1+x_2-1)



THE McCULLOCH-PITTS NEURON

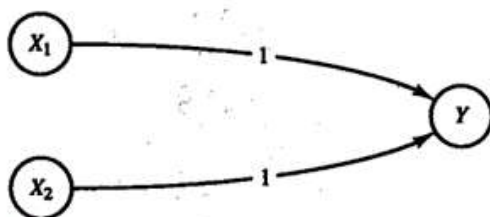
- The first formal definition of a synthetic neuron model based on the highly simplified considerations of the biological model described in the preceding section was formulated by McCulloch and Pitts (1943). The inputs x_i , for $i = 1, 2, \dots, n$, are 0 or 1, depending on the absence or presence of the input impulse at instant k . The neuron's output signal is denoted as o . The firing rule for this model is defined as follows

$$o^{k+1} = \begin{cases} 1 & \text{if } \sum_{i=1}^n w_i x_i^k \geq T \\ 0 & \text{if } \sum_{i=1}^n w_i x_i^k < T \end{cases}$$



Logic Functions

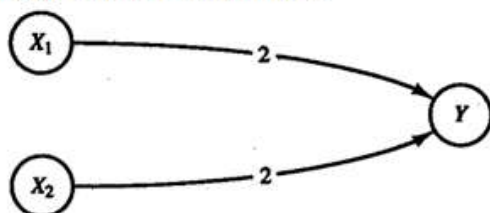
- AND
- The AND function gives the response "true" if both input values are "true"; otherwise the response is "false." If we represent "true" by the value 1, and "false" by 0, this gives the following four training input, target output pairs:



x_1	x_2	$\rightarrow$	y
1	1		1
1	0		0
0	1		0
0	0		0

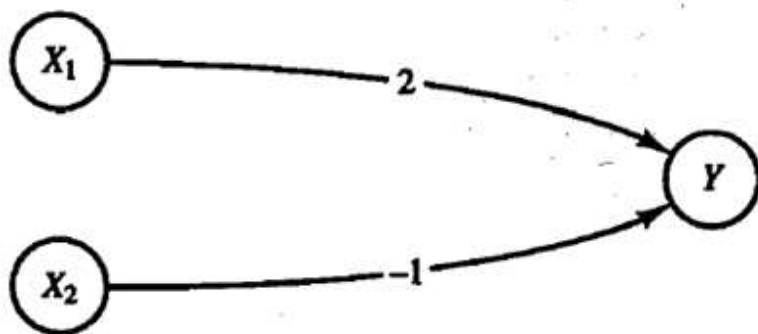
OR gate

The threshold on unit Y is 2.



x_1	x_2	$\rightarrow$	y
1	1		1
1	0		1
0	1		1
0	0		0

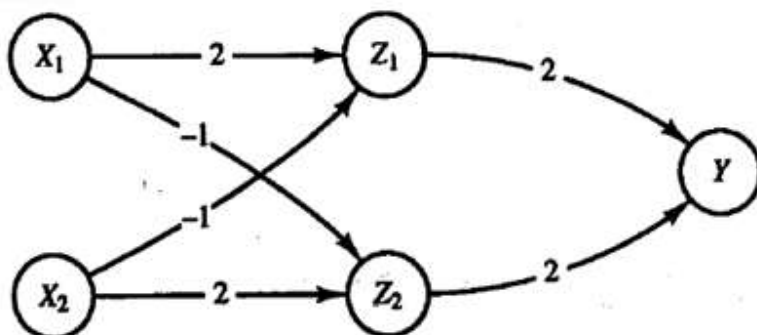
AND NOT



x_1	x_2	$\rightarrow y$
1	1	0
1	0	1
0	1	0
0	0	0

XOR

$$x_1 \text{ XOR } x_2 \leftrightarrow (x_1 \text{ AND NOT } x_2) \text{ OR } (x_2 \text{ AND NOT } x_1).$$



x_1	x_2	$\rightarrow y$
1	1	0
1	0	1
0	1	1
0	0	0

$$z_1 = x_1 \text{ AND NOT } x_2$$

$$z_2 = x_2 \text{ AND NOT } x_1.$$

$$y = z_1 \text{ OR } z_2.$$